

Module 5 — Secure Development Report

Karim Zaky | JHU Modern Software Concepts in Python

Installation and packaging

Run these commands from module_5 using Python 3.13. PostgreSQL must be running with the application database and permissions configured. Graphviz is installed separately (brew install graphviz on macOS). Live scraping requires macOS and Chrome.

pip + venv

```
python3.13 -m venv .venv-pip
.venv-pip/bin/python -m pip install -r requirements.txt
.venv-pip/bin/python -m pip install --no-deps -e .
.venv-pip/bin/python -m pip check
```

uv

```
uv venv --python python3.13 .venv-uv
uv pip install --python .venv-uv/bin/python -r requirements.txt
uv pip install --python .venv-uv/bin/python --no-deps -e .
uv pip check --python .venv-uv/bin/python
```

Copy .env.example to .env and enter the database credentials. Start the application with either command below, open <http://127.0.0.1:5000/analysis>, and stop it with Control+C.

```
.venv-pip/bin/gradcafe-module5
# Or:
.venv-uv/bin/gradcafe-module5
```

setup.py makes the project installable by defining its source modules, dependencies, Python requirement, and console entry point. Editable installs make imports consistent in local runs, tests, and CI while keeping source changes immediately available. Both installation methods were verified with Pylint 10.00/10, 167 passing tests, 100% statement coverage, and a working webpage. These instructions use editable installs; requirements specify compatible ranges rather than exact locked versions.

SQL injection defenses and LIMIT enforcement

Dynamic SQL is composed with `psycopg.sql.SQL`, table and column names use `sql.Identifier`, and values use placeholders with separate parameters. Identifier allowlists restrict inserts to the approved applicants table and columns. User values are never interpolated into SQL text; malicious record text is treated as data.

```
statement, params = build_count_statement(limit=1)
cursor.execute(statement, params)
```

```
statement = build_insert_statement()
cursor.executemany(statement, prepared_rows)
```

Builders construct statements separately from execution. Every application read query includes LIMIT. Requested limits are clamped to 1–100, malformed values become 1, scalar results are capped at one row, and top-university results at five. ORM reads apply corresponding limits. LIMIT caps returned rows; aggregate calculations still examine the relevant underlying data. INSERT and schema DDL do not use a SELECT-style LIMIT. Tests verify malicious inputs, identifier rejection, parameter binding, and bounded reads.

Database permissions and credentials

Credentials are read from environment variables or the ignored `.env` file; `.env.example` contains placeholders. The application connects to `gradcafe_module5` as `gradcafe_module5_app`, which is not the table owner and has no superuser, database creation, role creation, or RLS bypass privileges.

```
CREATE ROLE gradcafe_module5_app
  LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE
  NOINHERIT NOREPLICATION NOBYPASSRLS;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
REVOKE TEMPORARY ON DATABASE gradcafe_module5 FROM PUBLIC;
GRANT CONNECT ON DATABASE gradcafe_module5
  TO gradcafe_module5_app;
GRANT USAGE ON SCHEMA public TO gradcafe_module5_app;
GRANT SELECT, INSERT ON TABLE public.applicants
  TO gradcafe_module5_app;
GRANT USAGE ON SEQUENCE public.applicants_p_id_seq
  TO gradcafe_module5_app;
```

CONNECT and schema USAGE permit access to existing database objects. SELECT supports analysis; INSERT supports Pull Data; sequence USAGE generates IDs for new records. The app saves records and therefore is not read-only. Verification confirmed SELECT and INSERT access while CREATE TABLE, ALTER TABLE, DROP TABLE, UPDATE, and DELETE were denied. The insert probe was rolled back, leaving 30,040 records. Evidence is saved in `database_privileges.txt`.

Dependency graph: six-sentence summary

The dependency graph was generated using pydeps and Graphviz, starting from `src/app.py`. The application depends on Flask for its web interface, `orm_queries` for database analysis, and `scrape_manager` for coordinating data pulls. The scraping manager depends on `scrape` to collect records and `load_data` to insert them into PostgreSQL. SQLAlchemy supports the database models and ORM queries, while `python-dotenv` supplies environment configuration. The shared `sql_safety` module is used by the loader, models, query service, and scraping manager, centralizing SQL safety and read-limit helpers. Because the graph starts from the application entry point, it represents the application's import dependencies rather than every file in the project.

```
PYTHONPATH=src python -m pydeps src/app.py \
--noshow -T svg -o dependency.svg
```

GitHub Actions workflow

The repository-root `.github/workflows/ci.yml` runs on main pushes, pull requests, and manual triggers. Its four checks enforce Pylint `--fail-under=10`, generate and validate `dependency.svg` using `pydeps` and `Graphviz`, run `snky` test, and execute `pytest` with 100% coverage enforced by `pytest.ini`. A PostgreSQL 18 service supplies the disposable test database. `SNYK_TOKEN` is stored as a GitHub Actions secret. Failed checks or a Snyk timeout fail the workflow, and verification output is saved as an artifact.

The run for commit `c8efdce` succeeded in 1 minute 45 seconds. Its Snyk scan tested 53 dependencies and reported zero issues with no vulnerable paths found, so no remediation was required by that scan. The required screenshots are saved separately as `actions_success.png` and `snky-analysis.png`.

Module 5 Secure Development

[ci.yml](#)

Filter workflow runs



1 workflow run

Event Status Branch Actor

This workflow has a `workflow_dispatch` event trigger.

Run workflow

✓ Add Module 5 CI checks for lint graph security and tests

Module 5 Secure Development #1: Commit `c8efdce` pushed by `karimzaky`

main

2 minutes ago

1m 45s

